HelixQL:

1. Database Provisioning & Authentication

- **How it works:** The administrator launches HelixQL and securely enters the target database instance credentials (Host, Port, Database Name, Username, and Password) into the environment layer.
- **The Behind-the-Scenes Action:** HelixQL establishes an isolated connection pool. It uses this temporary access to sweep the system catalog once, mapping out the available database tables to create a local text blueprint file containing empty `CREATE TABLE` layout structures.

2. Natural Language Input

- **How it works:** The administrator interacts with the UI dashboard and types a normal business intelligence question.
- **Example:** *"Who made the most orders from Gujarat this month?"*

3. Isolated Metadata RAG (Context Assembly)

- **How it works:** The application intercepts the English text string locally. A deterministic parsing module extracts key keywords and references the local schema blueprint file built in Step 1.
- **The Data Shift:** It prunes away the entire database layout down to the absolute bare minimum, selecting only the empty schemas for the tables explicitly needed (e.g., `users` and `orders`). **Crucially, zero actual rows of corporate data are touched or processed here.**

4. Text-to-SQL Synthesis

- **How it works:** HelixQL bundles the localized, isolated schema definitions alongside the user's original English prompt into a system instruction packet and dispatches it to the LLM.
- **The Output:** The LLM translates the intent into a raw SQL code string targeted at the database's specific dialect syntax (e.g., MySQL 8.0).

5. The SQLGlot Abstract Syntax Tree Guardrail

- **How it works:** The raw code string returned by the AI is intercepted by a custom Python safety controller before it can open a network socket to the database.
- **The Code Logic:** The controller feeds the string into **SQLGlot**, transforming the text into a mathematical **Abstract Syntax Tree (AST)** programming object. The engine inspects the tree nodes: it strictly validates that the root operation is an instance of `sqlglot.exp.Select`. If a token matching `DROP`, `DELETE`, `TRUNCATE`, or `ALTER` is detected anywhere in the tree, the context is immediately dropped with a hard security violation exception.

6. Zero-Impact Execution Estimation (`EXPLAIN`)

- **How it works:** The sanitized query string is passed to the connection pool configured in Step 1. The engine automatically prepends the **`EXPLAIN`** command to the line (ensuring the performance-heavy `ANALYZE` modifier is completely left out).
- **The Performance Pass:** The database driver fires this command at the live server. Because it is a standard `EXPLAIN`, the database optimizer references its tiny internal performance metrics statistics tables instead of reading actual rows. It computes the abstract operational cost layout and returns the estimation results in **under 1 millisecond** with zero server overhead.

7. The Stateful Helix Feedback Loop (Conditional Error Interception)

- **How it works:** The system runs the execution pipeline inside a structured programming `try/except` monitoring block.
    - **Branch A (The Success Path):** If the `EXPLAIN` call resolves successfully with no errors, the system runs the actual safe query, extracts the raw performance metrics and target rows, and seamlessly forwards them to Step 8.
    - **Branch B (The Failure Path / Self-Correction):** If the database engine drops a runtime exception (e.g., the LLM hallucinated an invalid column name like `u.customer_id`), HelixQL intercepts the crash. It increments a local tracker matrix capped at a **maximum of 3 attempts**. It packages the broken query string along with the exact, raw database driver error trace log and routes it back to the synthesis layer. The agent reads the error, modifies its mistake, and outputs a self-healed query back to the Step 5 gate.

8. Human-Readable Synthesis (The Final Output)

- **How it works:** Once a verified, error-free query execution settles, the raw database rows and the performance cost logs are sent back up to the LLM one last time.
- **The Ultimate Delivery:** The LLM acts as an editor, taking the raw rows and translating them into a perfectly polished, elegant conversational summary for the user interface layout.

HelixQL Implementation Documentation

This document outlines the step-by-step deployment and operational phases required to build, secure, and integrate **HelixQL**'s complete multi-tier ecosystem.

🌐 Phase 1: Centralized Web SaaS Platform Implementation (Control Plane)

Build the user-facing web platform responsible for account onboarding, email verification, and user state management.

Step 1.1: Web Workspace & Database Migration

- Set up a web framework project directory (e.g., Next.js, React, or a standard web framework) separate from the desktop application.
- Initialize the centralized relational database using the 5-table schema design: `users`, `email_verification_logs`, `subscription_plans`, `user_subscriptions`, and `telemetry_logs`.

Step 1.2: Decentralized Signup & Email Verification Loop

- Develop the user registration endpoint. When a user signs up, the backend hashes their password securely, creates a deactivated user record, and generates a unique, time-sensitive verification uuid token stored in the verification log table.
- Integrate a background mailing service wrapper to automatically dispatch an activation email containing the unique verification URL to the user's email address.

Step 1.3: Token Generation & Verification Assertion

- Build the verification link handler. When clicked, the web server asserts that the token exists, is unused, and has not passed its expiration window.
- Upon successful assertion, flag the user account as verified and programmatically generate a long-lived, unique cryptographic **`api_token`** string tied to their profile.
- Automatically initialize a free-tier subscription link for the user to manage their monthly query allowances.

📂 Phase 2: Workspace Scaffolding & Environment Setup

Establish the native desktop environment and cloud gateway directories to prepare for data plane operations.

Step 2.1: Desktop & Gateway Directory Partitioning

- Create a `client-desktop/` folder for the native Electron desktop infrastructure.
- Create a `server-gateway/` folder for the Python FastAPI translation microservice.

Step 2.2: Gateway Environment Initialization

- Initialize a Python virtual environment inside the server directory to prevent global dependency conflicts.
- Install core packages required for high-performance async routing (`fastapi`, `uvicorn`), Abstract Syntax Tree parsing (`sqlglot`), and the official cloud language model SDK.

Step 2.3: Desktop Client Initialization

- Initialize a Node.js runtime manifest within the desktop folder.
- Install developer dependencies to manage the native operating system window process and chromium runtime shell.
- Install production database connectors locally to handle direct database streaming sockets over the internal network.

🔒 Phase 3: Server-Side Gateway Implementation (Python FastAPI)

Build the stateless cloud gateway responsible for shielding master API keys, validating user tokens, and verifying query structures.

Step 3.1: Secure Environment Variable Mapping

Create an air-gapped environment configuration file located **strictly** on the cloud gateway server. Store the master cloud language model credentials here so they remain entirely hidden from the downloadable client desktop binary.

Step 3.2: Token Authentication Middleware Interception

Implement an authentication middleware layer on the FastAPI router. Every incoming request from a desktop client must include the user's custom `api_token` in the request headers. The gateway queries the central web database to verify the token is valid and checks that the user's monthly query limit hasn't been exceeded before allowing the request to proceed.

Step 3.3: Programmatic AST Security Pipeline

Develop the query validation module to parse incoming text strings into logical mathematical tree nodes using `SQLGlot`.

- Enforce a rule asserting that the root expression of the generated query must be a data retrieval (`SELECT`) operation.
- Build a recursive walking function that inspects every sub-node in the tree. Reject the transaction immediately if data-mutating or schema-destructive tokens are discovered anywhere in the tree.

Step 3.4: Context-Insulated LLM Synthesis Engine

Set up the AI translation module using the cloud service library.

- Configure a rigid system instruction prompt forcing the language model to act purely as a raw code translator.
- Ensure the model parameter settings use a temperature of zero to force deterministic syntax outputs and remove creative phrasing variance.

💻 Phase 4: Client-Side Desktop App Implementation (Electron)

Build the native user interface and the local operating system driver that communicates safely within the enterprise firewall.

Step 4.1: Secure Desktop Login & Token Caching

- Design a login screen inside the Electron app. When the admin logs in with their web platform credentials, the desktop app fires an authentication request to your web SaaS server.
- Upon successful authentication, the web server returns the user's unique `api_token`. The Electron app stores this token securely in local client state memory to authenticate all future translation requests.

Step 4.2: Electron Main & Preload Process Configuration

- Configure the background Main process to instantiate the primary user display window and manage operating system resources.
- Implement an isolated Inter-Process Communication bridge script. This allows the user interface script to pass parameters to the local OS layer without exposing raw system access directly to the browser view.

Step 4.3: Local Database Socket Interceptor

Inside the Electron background process, configure an asynchronous listener to catch incoming pipeline requests. This process uses the locally input database credentials to spin up a native client connection over secure, internal network sockets.

Step 4.4: Decentralized Data Flow Routing

Wire the core operational logic inside the client application to follow this sequential path once a user triggers the analysis pipeline:

1. Dispatch a secure HTTPS request containing the English prompt, empty schema metadata, and the cached user `api_token` to the cloud gateway.
2. Receive the safe SQL code string back from the gateway.
3. Prepend a structural execution estimation command (`EXPLAIN`) to the query and run it against the target database to collect instant optimizer cost metrics without touching actual rows.
4. Execute the raw data retrieval statement separately to extract the transactional rows safely into local desktop memory.

Step 4.5: Frontend UI Dashboard Layout

Design a dual-panel dashboard using standard interface layouts.

- Group connection input fields together (Host, Port, User, Password) and route them to persist exclusively in local client memory.
- Set up a split-screen workspace module: render the resulting database records into a clean data table layout on one side, and dump the raw database optimizer execution cost logs onto the other side for administrative review.

Tech Stack:

1. Control Plane (Centralized Web SaaS)

Purpose: Handles user accounts, subscriptions, and licensing.

- **Frontend: React** or **Next.js** (Standard web interface for user registration and billing).
- **Database: MongoDB** * *Why:* Perfect for storing flexible user profiles, token expiration states, subscription records, and chronological query usage logs (`telemetry_logs`).

2. Data Plane (Client Desktop Application)

Purpose: Executes secure database queries locally inside corporate firewalls.

- **Shell Framework: Electron** (Wraps the application into a native Linux/Ubuntu executable).
- **UI Interface: Vite + React + Tailwind CSS** (Builds the lightweight admin metrics dashboard).
- **Native Drivers: Node-MySQL2** or **PG-Pool** (Handles direct, internal network socket streaming to client databases).

3. Server Gateway Tier (Cloud Microservice)

Purpose: Stateless language translation and syntax security guard.

- **Backend Framework: Python FastAPI** (Asynchronous, high-performance API endpoint handler).
- **Security Guardrail: SQLGlot** (Pure-Python AST compiler that intercepts and kills mutating queries).
- **AI Core Engine: Google GenAI SDK** (Interfaces with `gemini-2.5-flash` with temperature set to `0.0`).

Functional Requirements Document (FRD) // HelixQL Engine

This document defines the functional requirements, system behavior, and operational parameters for **HelixQL**. It outlines what the system must do to support user management, metadata isolation, secure translation, and local desktop execution.

🛠️ Functional Requirements Overview

1. Control Plane: Centralized Web SaaS (MongoDB User Management)

- **FR-1.1: User Registration & Hashing:** The web platform must allow users to register with a name, email, and password. Passwords must be cryptographically hashed and stored securely as documents within a MongoDB users collection.
- **FR-1.2: Asynchronous Email Verification:** Upon registration, the system must generate a unique, time-restricted verification token, save it to an email_verification_logs collection, and dispatch an activation link via an email automation service. The user account status must remain inactive until verified.
- **FR-1.3: Verification Assertion & Activation:** When a user accesses the verification URI, the server must validate the token's lifecycle against an expiration window. Upon a valid check, the user's document status must update to active.
- **FR-1.4: Cryptographic api_token Generation:** On account activation, the web tier must programmatically generate a long-lived, unique cryptographic api_token string and assign it to the user's MongoDB document.
- **FR-1.5: Plan & Allowance Initialization:** The system must attach a default subscription sub-document or reference linked to a 'Free Tier' plan profile, tracking a hard query allowance limit per month.

2. Data Plane: Electron Desktop Client (Local Environment)

- **FR-2.1: Native Cross-Platform Shell:** The desktop client must be compiled as a native executable running flawlessly inside Linux (Ubuntu), Windows, and macOS host operating systems.
- **FR-2.2: Localized Account Authentication:** The app must provide a login interface. Upon entering credentials, it must dispatch an encrypted request to the web SaaS portal to authenticate the session and pull down the user's matching api_token.
- **FR-2.3: Secure Token Caching:** The app must cache the retrieved api_token strictly in local client state or volatile runtime memory. It must never save this token to a public directory.
- **FR-2.4: Local Connection String Isolation:** The app must expose input configurations to collect database connection arrays (Host, Port, User, Password). These configuration values must be persisted exclusively in local client memory on the machine and must never cross network perimeters to the cloud.
- **FR-2.5: Structural Metadata Extraction (Local RAG):** When a user inputs an analytical request, the desktop runtime must sweep the local target database's system catalog natively. It must match conversational tokens against the schema layout to extract **only** empty structural table configurations (CREATE TABLE schema scripts), dropping unrelated structural mappings entirely.
- **FR-2.6: Dual-Panel Dashboard Rendering:** The user interface must feature a bifurcated design panel: one view dedicated to displaying the resulting database data records in an interactive grid, and an adjacent view rendering the database optimizer metrics and cost score metrics.

3. Server Gateway Tier: Python FastAPI (AI Processing Layer)

- **FR-3.1: Token-Authenticated Endpoint Interception:** The FastAPI cloud gateway must enforce an authentication barrier on the translation route. It must extract the incoming api_token header, check its status against the central MongoDB cluster, and reject unauthorized requests with a status code 401/403.
- **FR-3.2: Usage Metering & Throttling:** Prior to invoking the AI pipeline, the gateway must verify the user's monthly query footprint. If the limit is within budget, it must increment their running usage total inside the tracking logs; otherwise, it must throw a query-limit exceeded exception.
- **FR-3.3: Pure-Syntax Prompt Assembly:** The gateway must format the system instructions dynamically by combining the user's English string and the anonymous structural metadata received from the client. It must explicitly command the LLM to output raw code with zero conversational text or markdown blocks.
- **FR-3.4: Programmatic Abstract Syntax Tree (AST) Validation:** The gateway must intercept the raw code string returned by the language model and feed it into a strict SQLGlot compilation validator before returning it to the user interface.
- **FR-3.5: Destructive Action Interception:** The AST validator must walk through every single query expression node. It must assert that the root operation is strictly a data retrieval instance (SELECT). If any modifying or destructive database actions (DROP, DELETE,

UPDATE, INSERT, ALTER) appear anywhere in the tree branches, it must immediately terminate the thread and pass a security violation log down to the client dashboard.

4. Direct Local Execution & Resilient Feedback

- **FR-4.1: Cost Estimation Prefixes (EXPLAIN Interceptor):** When the Electron client process receives a safe SQL string from the gateway, it must programmatically prepend the **EXPLAIN** command line to the query string.
- **FR-4.2: Zero-Footprint Telemetry Execution:** The client app must dispatch this EXPLAIN query block directly to the live database instance via its native OS networking socket. It must collect the optimizer's abstract execution plan tree metrics in under 1 millisecond without locking rows or reading raw database records.
- **FR-4.3: Asynchronous Data Hydration:** If the EXPLAIN call executes smoothly without errors, the app must strip the prefix, fire the clean SELECT command natively to extract the actual database rows, and flush both data sets up to the interface layout.
- **FR-4.4: Catching Exceptions:** The database driver execution routine inside the Electron client must be wrapped inside a structured error monitoring block to catch native runtime exceptions (such as identifier hallucinations or join constraints).
- **FR-4.5: Automated Refinement Looping:** If a database error occurs during the EXPLAIN check, the client process must block the error from breaking the application view. It must increment a local loop counter capped at a **maximum of 3 attempts**. It must bundle the incorrect SQL string along with the raw runtime exception log returned by the database driver, and dispatch it back to the gateway to initiate automated refinement and self-healing.

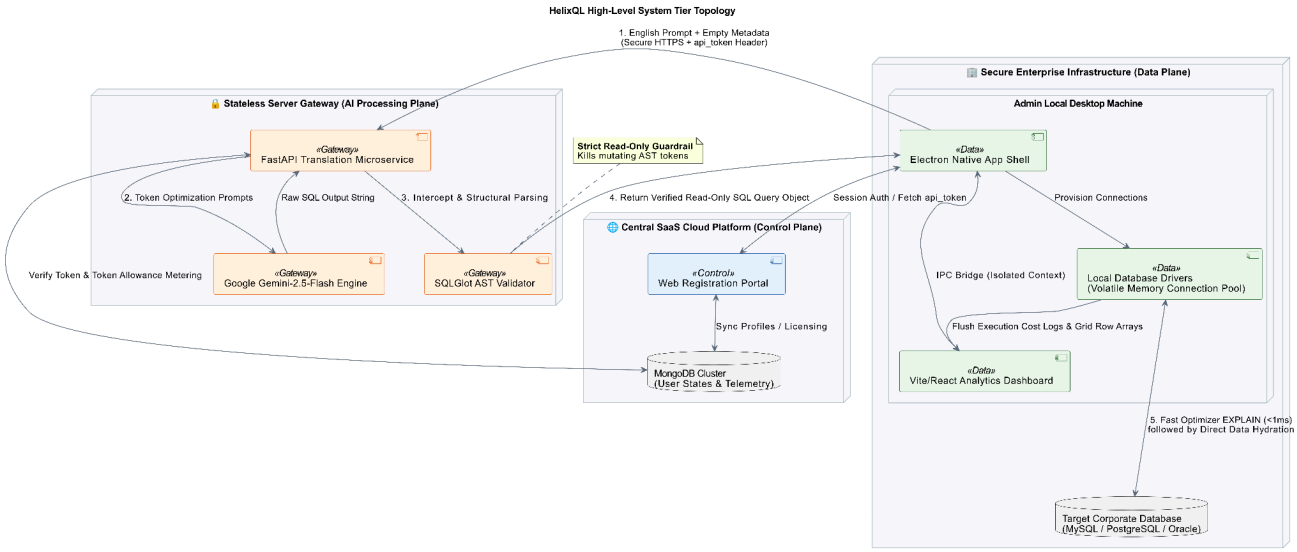
📈 System Operational Lifecycle Matrix

Triggers	Source / Component	Actions Performed	Final Outputs
User Sign-up	Centralized Web Portal	Hashes password; populates MongoDB; dispatches token link via email service.	Account pending state document in MongoDB.

App Launch	Electron Client Interface	Collects login; validates against web database; fetches and caches client api_token.	Securely authorized local desktop user session.
Pipeline Run	Dashboard UI View	Extracts empty database layout structures locally; ships text prompt and empty schema metadata via HTTPS to gateway.	Encrypted translation payload dispatch.
Translation Pass	FastAPI Gateway Server	Validates api_token; generates SQL code via LLM; compiles AST tree and walks nodes for security threats.	Sanitized, read-only SQL string query object.
Query Intercept	Electron OS Background	Prepends EXPLAIN; executes against local database socket; checks for native exceptions.	Abstract execution cost logs map block.
On Success	Electron OS Background	Executes raw SELECT query string via local connection stream; pipes arrays to UI.	Hydrated data table grid + metric diagnostics.

On Error	Electron Feedback Loop	Intercepts database driver crash; tracks counter limit; routes error stack logs back to cloud for automated self-healing.	Corrected SQL generation block re-evaluation.

Architecture Diagram:



Flow Diagram: